PROJECT REPORT

DECLARATION

I, Rangeet Dutta, hereby declare that the project report titled “Machine Learning and Deep Learning Projects Portfolio” is a record of the work carried out by me during my internship from January 1, 2026 to April 1, 2026 at CloudCredits.

I further declare that this report has been submitted in partial fulfillment of the requirements for my training, and that the work presented here is original and has not been submitted elsewhere for any other purpose.

I take full responsibility for the authenticity of the data and results presented in this report.

ACKNOWLEDGEMENT

I would also like to thank my mentors and peers for their continuous support and encouragement during the completion of this project.

Lastly, I would like to express my gratitude to everyone who directly or indirectly contributed to the successful completion of this report.

ABSTRACT

This report presents a comprehensive overview of multiple projects carried out in the domain of Machine Learning and Deep Learning during the internship period. The projects include Boston Housing Price Prediction, Handwritten Digit Classification using CNN (MNIST), Stock Price Prediction using LSTM, and Diabetes Prediction using K-Nearest Neighbors (KNN).

The work involves data collection, preprocessing, exploratory data analysis (EDA), and implementation of various machine learning and deep learning models. Techniques such as Random Forest, Decision Tree, Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and GridSearchCV for hyperparameter tuning have been applied.

The objective of these projects is to gain practical exposure to real-world datasets, understand underlying patterns through data analysis, and build predictive models with improved accuracy. The results demonstrate the effectiveness of different algorithms across regression, classification, and time-series forecasting tasks.

This report highlights the importance of data preprocessing, feature analysis, and model optimization in achieving reliable and accurate predictions.

Boston Housing

Objective

The objective of this project is to analyze the Boston Housing dataset and build machine learning models to predict house prices based on various features such as crime rate, number of rooms, and socio-economic conditions.

Dataset

Dataset: Boston Housing Dataset (Kaggle)

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154	18.455534	356.674032	12.653063
	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537116	2.164946	91.294864	7.141062
	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000	1.730000
	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.000000	17.400000	375.377500	6.950000
	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000	19.050000	391.440000	11.360000
	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.000000	20.200000	396.225000	16.955000
	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000	37.970000

Records: 506 observations

Features: 13 input features + 1 target variable

The dataset includes important variables like crime rate (CRIM), number of rooms (RM), and lower-income population percentage (LSTAT), which influence housing prices.

Data Preprocessing

Loaded dataset using pandas Checked dataset shape, columns, and summary statistics Verified presence of missing values (dataset is mostly clean)

```
print("--- Duplicate Rows Check ---")
duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")

if duplicates > 0:
    df = df.drop_duplicates()
    print("✓ Duplicates removed.")
else:
    print("✓ No duplicates found.")
```

```
--- Duplicate Rows Check ---
Number of duplicate rows: 0
✓ No duplicates found.
```

Defined input features (X) and target variable (MEDV) Split data into training and testing sets

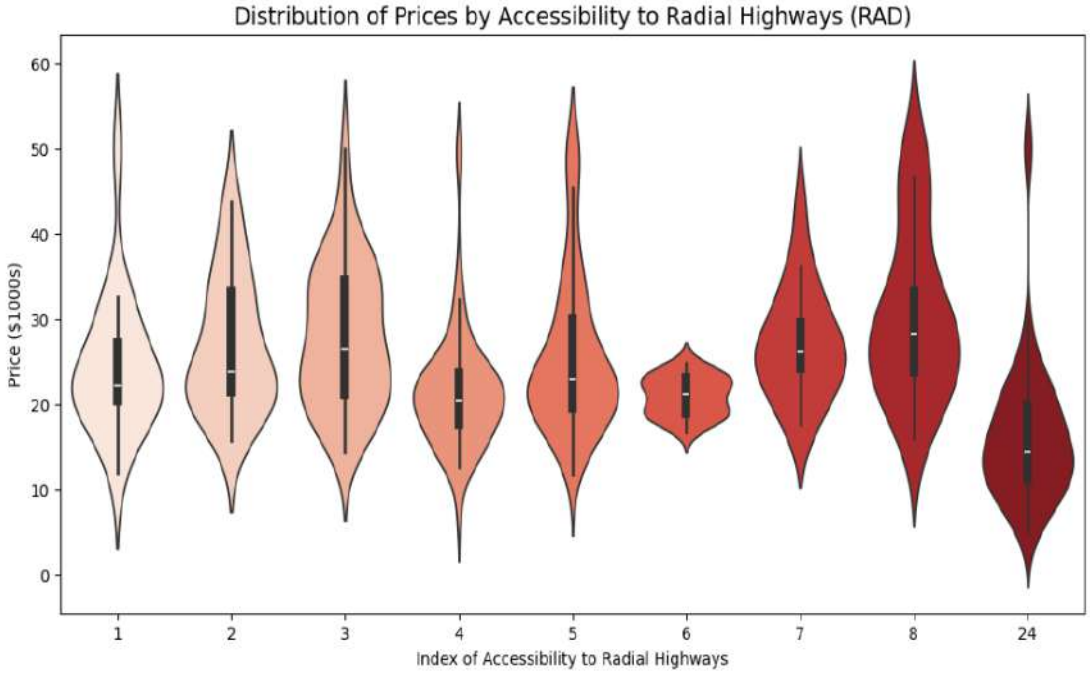
Exploratory Data Analysis (EDA)

Used .describe() to understand statistical distribution Visualized relationships between features and target

In [17]:

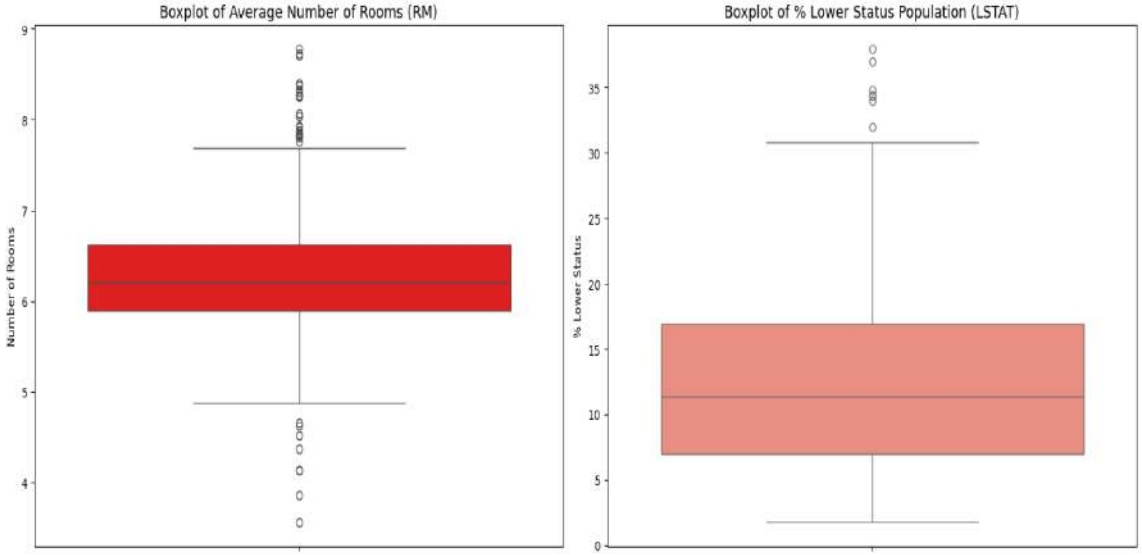
```
plt.figure(figsize=(12, 6))
sns.violinplot(x='RAD', y='MEDV', data=df, palette='Reds')
plt.title('Distribution of Prices by Accessibility to Radial Highways (RAD)', fontsize=14)
plt.xlabel('Index of Accessibility to Radial Highways')
plt.ylabel('Price ($1000s)')
plt.show()
```

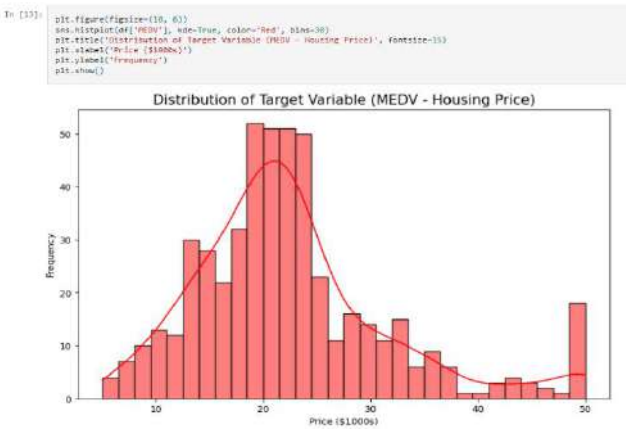
```
C:\rasa_tmp\ipykernel_11368\1126971448.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False`

  sns.violinplot(x='RAD', y='MEDV', data=df, palette='Reds')
```

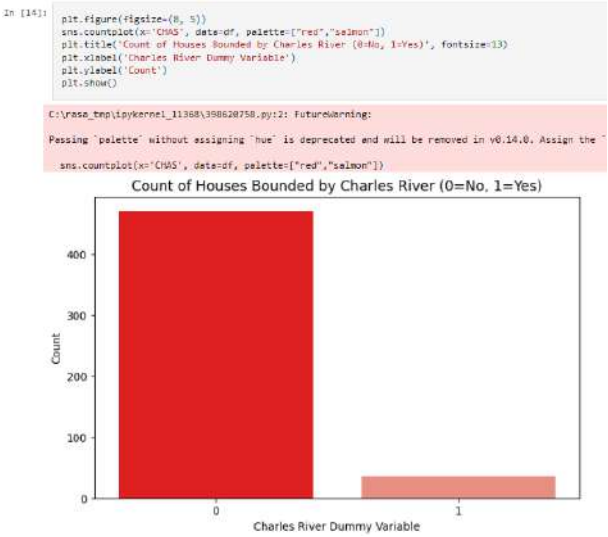


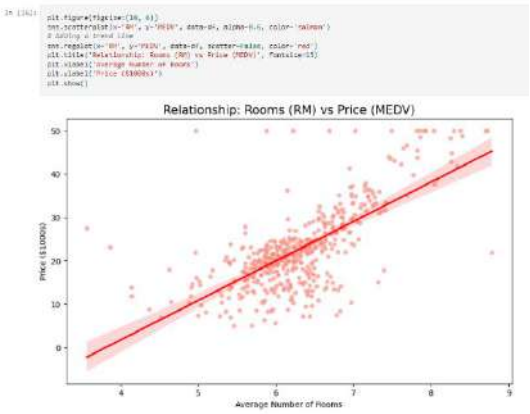
In [15]:

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.boxplot(y=df['RM'], ax=axes[0], color='red')
axes[0].set_title('Boxplot of Average Number of Rooms (RM)', fontsize=13)
axes[0].set_ylabel('Number of Rooms')

sns.boxplot(y=df['LSTAT'], ax=axes[1], color='salmon')
axes[1].set_title('Boxplot of % Lower Status Population (LSTAT)', fontsize=13)
axes[1].set_ylabel('% Lower Status')

plt.tight_layout()
plt.show()
```









Analyzed key features such as:

- RM (number of rooms)
- LSTAT (lower-income population %)
- PTRATIO (student-teacher ratio)

Key Insights:

- Higher number of rooms (RM) → higher house prices
- Higher LSTAT → lower house prices
- Socio-economic factors strongly impact pricing

Feature Selection

Focused on important variables influencing price:

- RM
- LSTAT
- PTRATIO

These features showed strong relationships with the target variable (MEDV).

Model Building

Models Used:

- Decision Tree Regressor
- Random Forest Regressor

In [44]:
```
rf = RandomForestRegressor(random_state=42)

param_grid_rf = {
    'n_estimators': [50, 100],
    'max_depth': [10, 20, None],
    'min_samples_leaf': [1, 4]
}

print("Training Random Forest with GridSearchCV...")
grid_rf = GridSearchCV(
    rf,
    param_grid_rf,
    cv=3,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
grid_rf.fit(X_train, y_train)
```

```
Training Random Forest with GridSearchCV...
```

Out[44]:
```
GridSearchCV(cv=3, estimator=RandomForestRegressor(random_state=42), n_jobs=-1,
             param_grid={'max_depth': [10, 20, None],
                         'min_samples_leaf': [1, 4],
                         'n_estimators': [50, 100]},
             scoring='neg_mean_squared_error')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [43]:
```
from sklearn.ensemble import RandomForestRegressor
```

In [45]:
```
best_rf = grid_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)

# Calculate metrics
r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

# Store results
model_results.append({
    'Model': 'Random Forest (Tuned)',
    'R2 (Test)': r2_rf,
    'RMSE (Test)': rmse_rf
})

print("\n Random Forest trained successfully!")
print(f"  Best Parameters: {grid_rf.best_params_}")
print(f"  R² Score: {r2_rf:.4f}")
print(f"  RMSE: {rmse_rf:.4f}")
```

```
Random Forest trained successfully!
 Best Parameters: {'max_depth': 10, 'min_samples_leaf': 1, 'n_estimators': 100}
 R² Score: 0.8920
 RMSE: 2.8141
```

In [40]:
```
dt = DecisionTreeRegressor(random_state=42)

param_grid_dt = {
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}

print("Training Decision Tree with GridSearchCV...")
grid_dt = GridSearchCV(
    dt,
    param_grid_dt,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)
grid_dt.fit(X_train, y_train)
```

```
Training Decision Tree with GridSearchCV...
```

Out[40]:
```
GridSearchCV(cv=5, estimator=DecisionTreeRegressor(random_state=42), n_jobs=-1,
             param_grid={'max_depth': [3, 5, 10, None],
                         'min_samples_leaf': [1, 5, 10],
                         'min_samples_split': [2, 10, 20]},
             scoring='neg_mean_squared_error')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [39]:
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV
```

In [41]:
```
best_dt = grid_dt.best_estimator_
y_pred_dt = best_dt.predict(X_test)

# Calculate metrics
r2_dt = r2_score(y_test, y_pred_dt)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

# Store results
model_results.append({
    'Model': 'Decision Tree (Tuned)',
    'R2 (Test)': r2_dt,
    'RMSE (Test)': rmse_dt
})

print("\n Decision Tree trained successfully!")
print(f"  Best Parameters: {grid_dt.best_params_}")
print(f"  R² Score: {r2_dt:.4f}")
print(f"  RMSE: {rmse_dt:.4f}")
```

```
Decision Tree trained successfully!
 Best Parameters: {'max_depth': 5, 'min_samples_leaf': 10, 'min_samples_split': 2}
 R² Score: 0.7827
 RMSE: 3.9510
```

Steps:

- Trained models using training data
- Made predictions on test data
- Compared performance of both models

Model Evaluation

Evaluated using:

- R^2 Score
- Mean Squared Error (MSE)

Observations:

- Random Forest Regressor performed better due to ensemble learning
- Decision Tree showed signs of overfitting

Results

Successfully predicted housing prices with good accuracy

Random Forest provided more stable and accurate predictions compared to Decision Tree

Conclusion

The project demonstrated how machine learning can be used for regression problems

Feature analysis helped identify key factors affecting housing prices

Ensemble methods like Random Forest improve prediction performance

Future Scope

Apply advanced models like XGBoost or Gradient Boosting

Perform hyperparameter tuning

Deploy model as a web application

Handwritten Digit Classification

Objective

The objective of this project is to build a deep learning model using Convolutional Neural Networks (CNN) to accurately classify handwritten digits (0–9) from image data.

Dataset

Dataset: MNIST Handwritten Digits

Total images: 70,000

Training: 60,000

Testing: 10,000

Image size: 28 × 28 pixels (grayscale)

The dataset consists of labeled images representing digits from 0 to 9 and is widely used as a benchmark for image classification tasks.

Data Augmentation

Applied transformations such as:

```
In [6]: Y_train = train["label"]
        X_train = train.drop(labels = ["label"],axis = 1)
        X_train = X_train / 255.0
        X_test = test / 255.0
        X_train = X_train.values.reshape(-1,28,28,1)
        X_test = X_test.values.reshape(-1,28,28,1)
        Y_train = to_categorical(Y_train, num_classes = 10)
```

- Rotation
- Zoom
- Width & height shifting

This helps:

- Increase dataset diversity
- Reduce overfitting
- Improve model generalization

Model Building (CNN)

Architecture:

Convolutional Layers (feature extraction)

MaxPooling Layers (downsampling)

Flatten Layer

Dense (Fully Connected) Layers

Output Layer (Softmax for classification)

CNN is highly effective for image tasks as it automatically learns spatial features from images.

```
In [8]: from tensorflow.keras import layers, models

        nets = 2   # reduced from 15
        models_list = []

        for _ in range(nets):
            m = models.Sequential([
                layers.Input(shape=(28,28,1)),

                # Block 1
                layers.Conv2D(16, 3, activation='relu'),
                layers.Conv2D(16, 3, activation='relu'),
                layers.MaxPooling2D(),
                layers.Dropout(0.25),

                # Block 2
                layers.Conv2D(32, 3, activation='relu'),
                layers.MaxPooling2D(),
                layers.Dropout(0.25),

                # Dense
                layers.Flatten(),
                layers.Dense(64, activation='relu'),
                layers.Dropout(0.3),
                layers.Dense(10, activation='softmax')
            ])

            m.compile(
                optimizer='adam',
                loss='categorical_crossentropy',
                metrics=['accuracy']
            )

            models_list.append(m)
```

Model Training

Compiled model with:

Loss function: categorical crossentropy

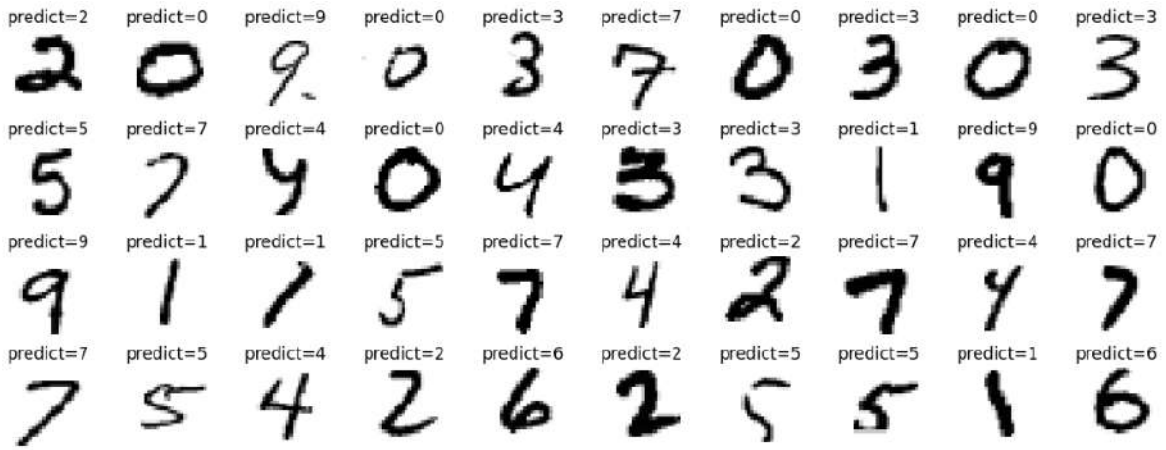
Optimizer: Adam

Trained model on augmented dataset

Used validation data to monitor performance

Visualization

Sample digit images



Conclusion

Successfully implemented CNN for image classification

Data augmentation improved model robustness

CNN proved highly effective for handwritten digit recognition tasks

Future Scope

Use advanced architectures (LeNet, ResNet)

Deploy model as a web app

Extend to more complex datasets (e.g., CIFAR-10)

Diabetes Prediction

Objective

The objective of this project is to analyze medical data and build a machine learning model to predict whether a patient is diabetic or not using classification techniques.

Dataset

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Dataset: Pima Indians Diabetes Dataset

Total records: 768 patients

Features: 8 medical attributes + 1 target variable

The dataset contains diagnostic measurements such as glucose level, BMI, insulin, and age to predict diabetes outcome (0 = non-diabetic, 1 = diabetic)

Data Description

Important features include:

- Glucose – Blood sugar level
- BMI – Body Mass Index
- Age – Patient age
- Insulin – Insulin level
- Pregnancies – Number of pregnancies

Outcome – Target variable (diabetes or not)

These features are key indicators for diabetes prediction.

Data Preprocessing

Loaded dataset using pandas

Checked for missing or invalid values (zero values in some columns)

Handled data inconsistencies

Split dataset into training and testing sets

Exploratory Data Analysis (EDA)

Used statistical summaries (.describe())

Out[3]:

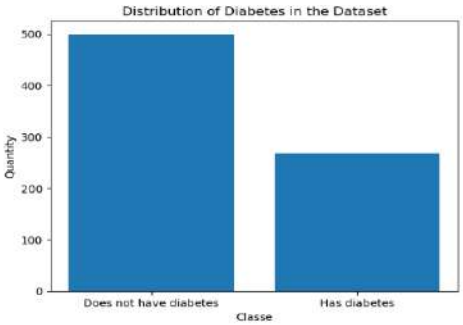
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.686763	72.405184	29.153420	155.548223	32.457464	0.471876	33.240885	0.348958
std	3.369578	30.435949	12.096346	8.790942	85.021108	6.875151	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	25.000000	121.500000	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.202592	29.153420	155.548223	32.400000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	155.548223	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

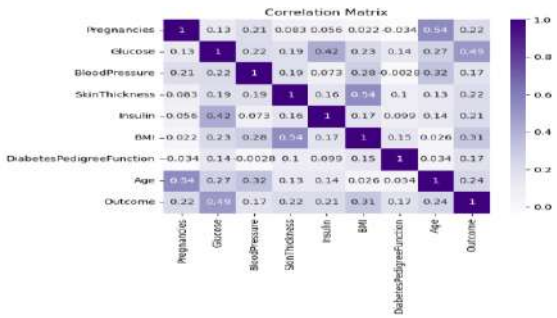
Visualized feature distributions

Analyzed correlations between features

Key Insights:

- Higher glucose levels strongly indicate diabetes
- BMI and age are important contributing factors





Dataset shows slight class imbalance (more non-diabetic cases)

Model Building

Model Used:

K-Nearest Neighbors (KNN)

Approach:

Trained KNN model on training data

Selected optimal number of neighbors using tuning

Hyperparameter Tuning

Applied GridSearchCV to find the best parameters

Tuned:

Number of neighbors (K)

Distance metrics

```
In [14]: from sklearn.pipeline import Pipeline
         from sklearn.preprocessing import StandardScaler
         from sklearn.neighbors import KNeighborsClassifier
         from sklearn.model_selection import GridSearchCV

         # Pipeline: scaling + KNN
         pipeline = Pipeline([
             ('scaler', StandardScaler()),
             ('knn', KNeighborsClassifier())
         ])

         # Hyperparameter grid
         param_grid = {
             'knn__n_neighbors': [3, 5, 7, 9],
             'knn__weights': ['uniform', 'distance'],
             'knn__metric': ['euclidean', 'manhattan']
         }

         # GridSearch with pipeline
         grid_cv_scale = GridSearchCV(
             pipeline,
             param_grid,
             cv=3,
             scoring='accuracy'
         )

         # Train
         grid_cv_scale.fit(X_train, y_train)

         # Best results
         print("Best parameters:", grid_cv_scale.best_params_)
         print("Best CV score:", grid_cv_scale.best_score_)

         # Evaluate on test set
         test_score = grid_cv_scale.score(X_test, y_test)
         print("Test accuracy:", test_score)

         Best Parameters: {'knn__metric': 'manhattan', 'knn__n_neighbors': 7, 'knn__weights': 'uniform'}
         Best CV Score: 0.7509960141064401
         Test Accuracy: 0.7273727172727273
```

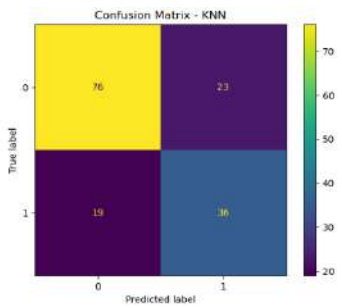
This improved model accuracy and reduced overfitting.

Model Evaluation

Evaluated using:

- Accuracy score
- Confusion matrix

Results:

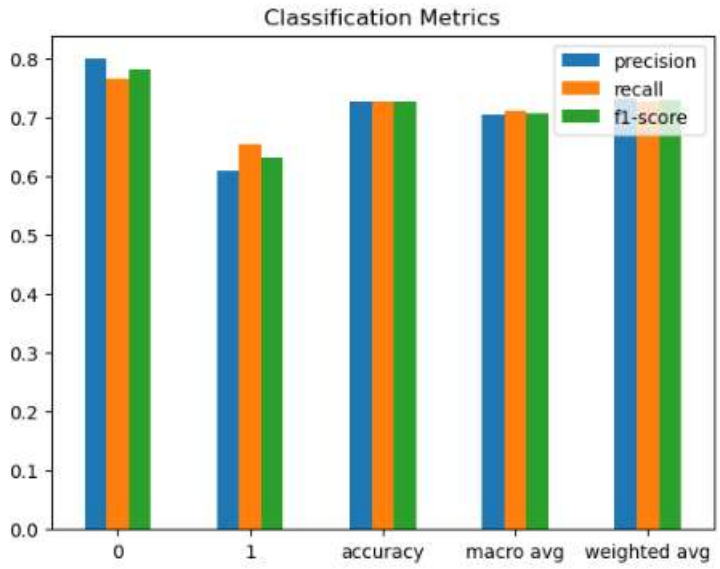


Tuned KNN model performed better than default model

Achieved good classification performance for diabetes prediction

Visualization

Classification matric



Conclusion

Successfully built a classification model for diabetes prediction

EDA helped identify important health indicators like glucose and BMI

GridSearchCV significantly improved model performance

Future Scope

Use advanced models (Random Forest, XGBoost)

Handle class imbalance using SMOTE

Deploy model as a healthcare prediction system

Stock Price Prediction

Objective

The objective of this project is to analyze historical stock price data and build a deep learning model using LSTM (Long Short-Term Memory) to predict future stock prices based on past trends.

Data Collection

Data collected using the yfinance library

Companies used:

- Apple (AAPL)
- Google (GOOG)
- Microsoft (MSFT)
- Amazon (AMZN)

Out[3]:

Price	Close	High	Low	Open	Volume	company_name	Close	High	Low	Open	...	Close	High	Low	Open	Volume	Close	High	Low	Open	Volume
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL		GOOG	GOOG	GOOG	GOOG	...	MSFT	MSFT	MSFT	MSFT	MSFT	AMZN	AMZN	AMZN	AMZN	AMZN
Date																					
2026-01-20	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	231.000000	235.089996	229.339996	233.759995	47737900.0
2026-01-21	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	231.309998	232.300003	226.880005	231.089996	47276100.0
2026-01-22	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	234.339996	235.720001	230.899994	234.050003	31913300.0
2026-01-23	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	239.160004	240.449997	234.570007	234.960007	33778500.0
2026-01-26	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	238.419998	240.949997	237.539993	239.979996	32825500.0
2026-01-27	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	244.679993	244.880005	238.080002	239.690002	38029200.0
2026-01-28	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	243.009995	247.779999	241.529999	246.369995	40882700.0
2026-01-29	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	241.729996	243.000000	236.740005	242.820007	47229600.0
2026-01-30	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	239.300003	243.320007	237.639999	239.889999	46535400.0
2026-02-02	NaN	NaN	NaN	NaN	NaN	AMAZON	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	243.975006	245.629898	238.169998	238.309998	20740379.0

10 rows × 21 columns

Time range: Last 1 year of historical data

The dataset includes stock attributes such as Open, High, Low, Close prices, and Volume.

Data Preprocessing

Combined data from multiple companies into a single DataFrame

Focused on closing prices for prediction

Normalized data for better model performance

Converted time-series data into sequences suitable for LSTM

Split dataset into training and testing sets

Exploratory Data Analysis (EDA)

Visualized stock price trends over time

Compared performance of multiple companies

Analyzed volatility and price fluctuations

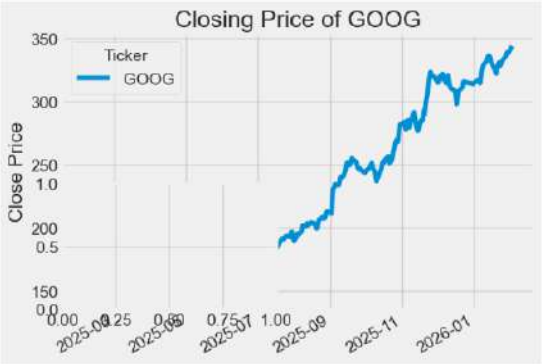
Key Insights:

- Stock prices show clear trends and fluctuations over time
- Different companies exhibit different growth patterns
- Time-series nature of data makes it suitable for sequence models

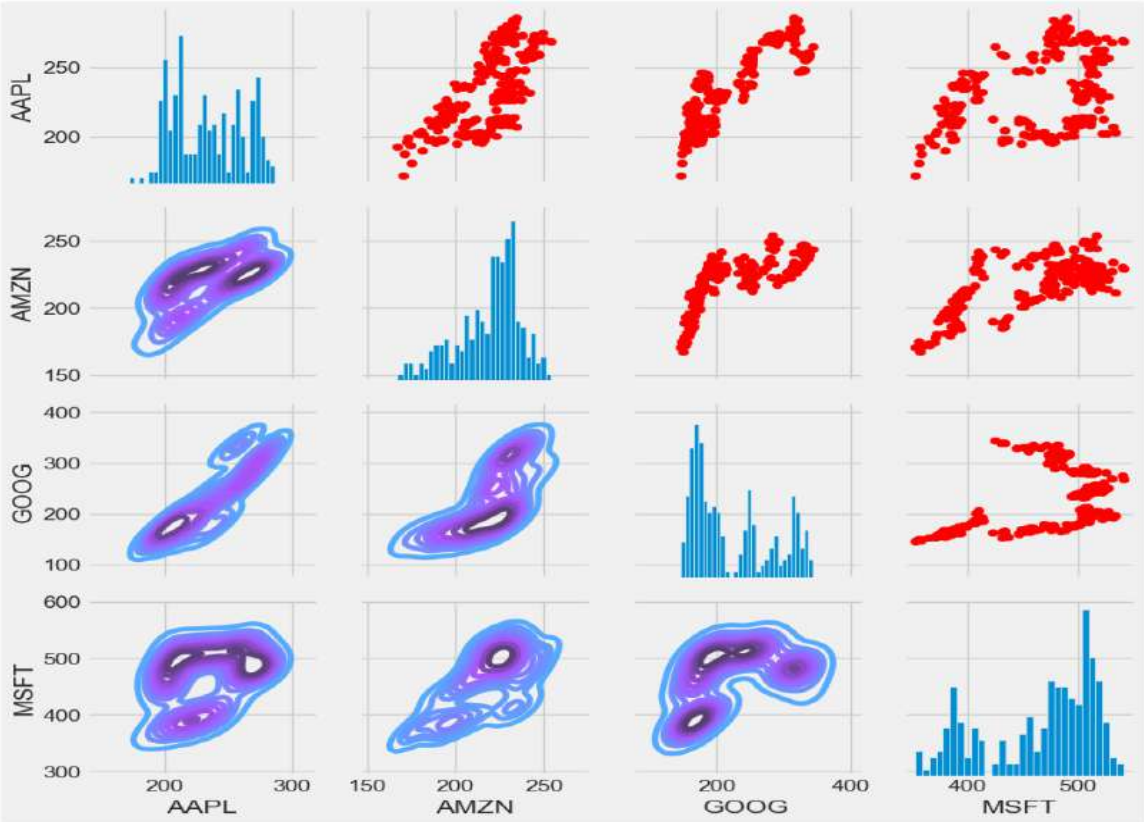
LSTM models are effective for such sequential data because they can capture long-term dependencies and trends

Closing Price of AAPL
Ticker
AAPL
Close Price
280
260
240
220
200
180
2025-03
2025-05
2025-07
2025-09
2025-11
2026-01

Closing Price of MSFT
Ticker
MSFT
Close Price
550
500
450
400
350
2025-03
2025-05
2025-07
2025-09
2025-11
2026-01

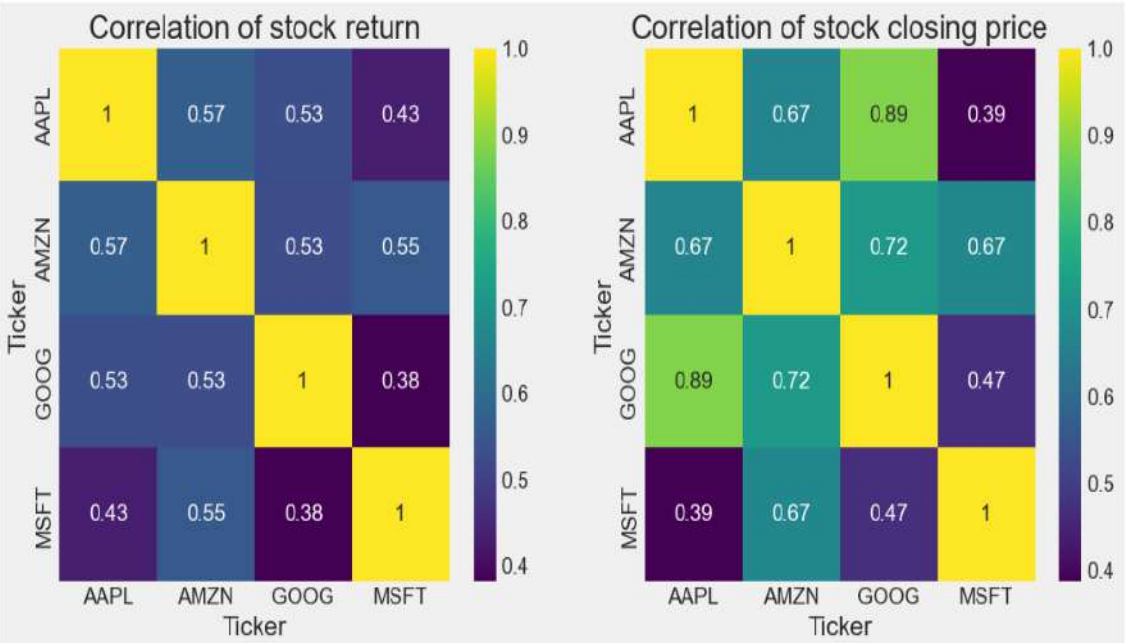

Closing Price of GOOG
Ticker
GOOG
Close Price
350
300
250
200
150
2025-03
2025-05
2025-07
2025-09
2025-11
2026-01

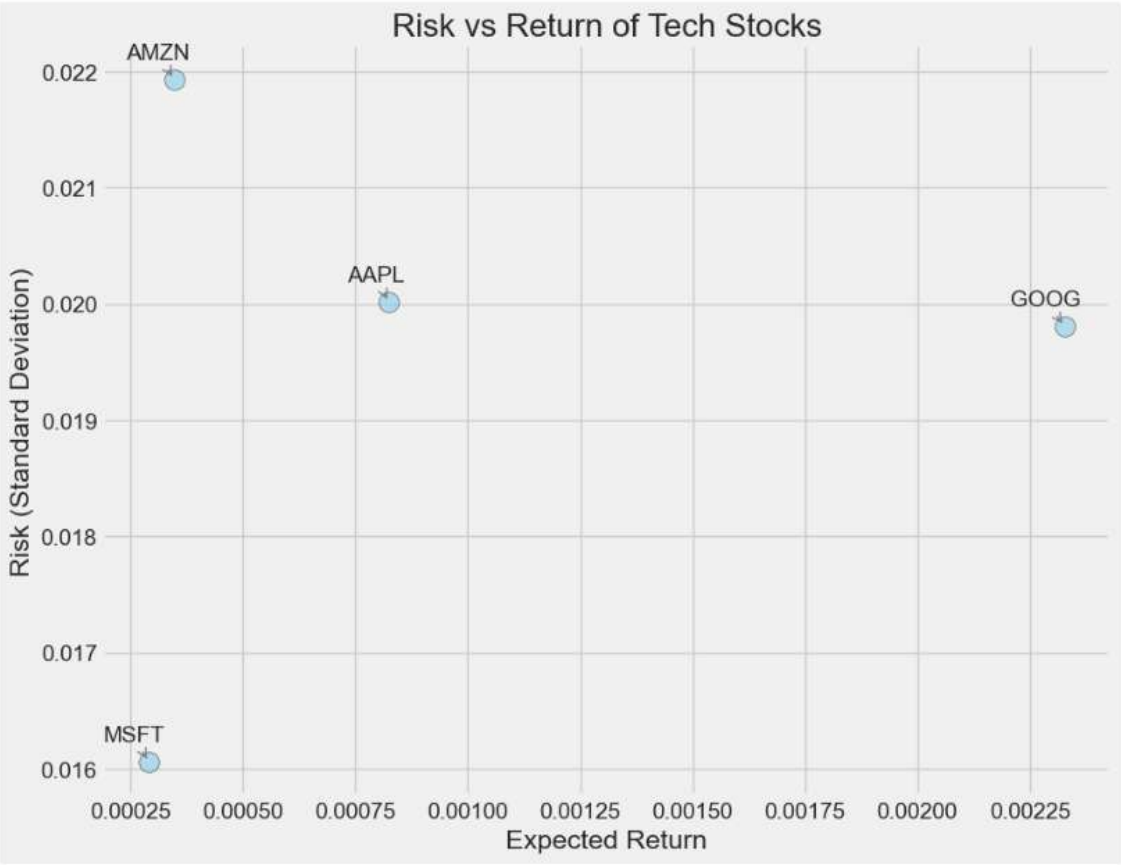
Closing Price of AMZN
Ticker
AMZN
Close Price
240
220
200
180
2025-03
2025-05
2025-07
2025-09
2025-11
2026-01

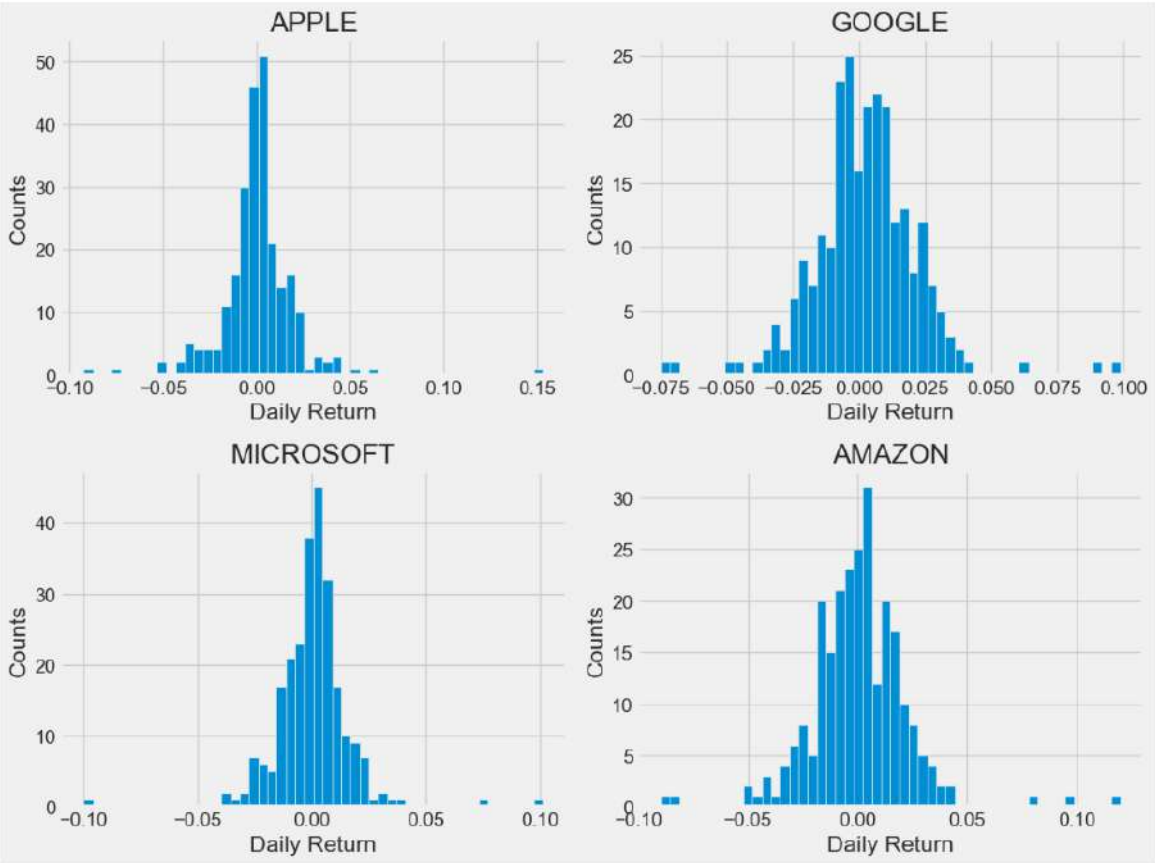


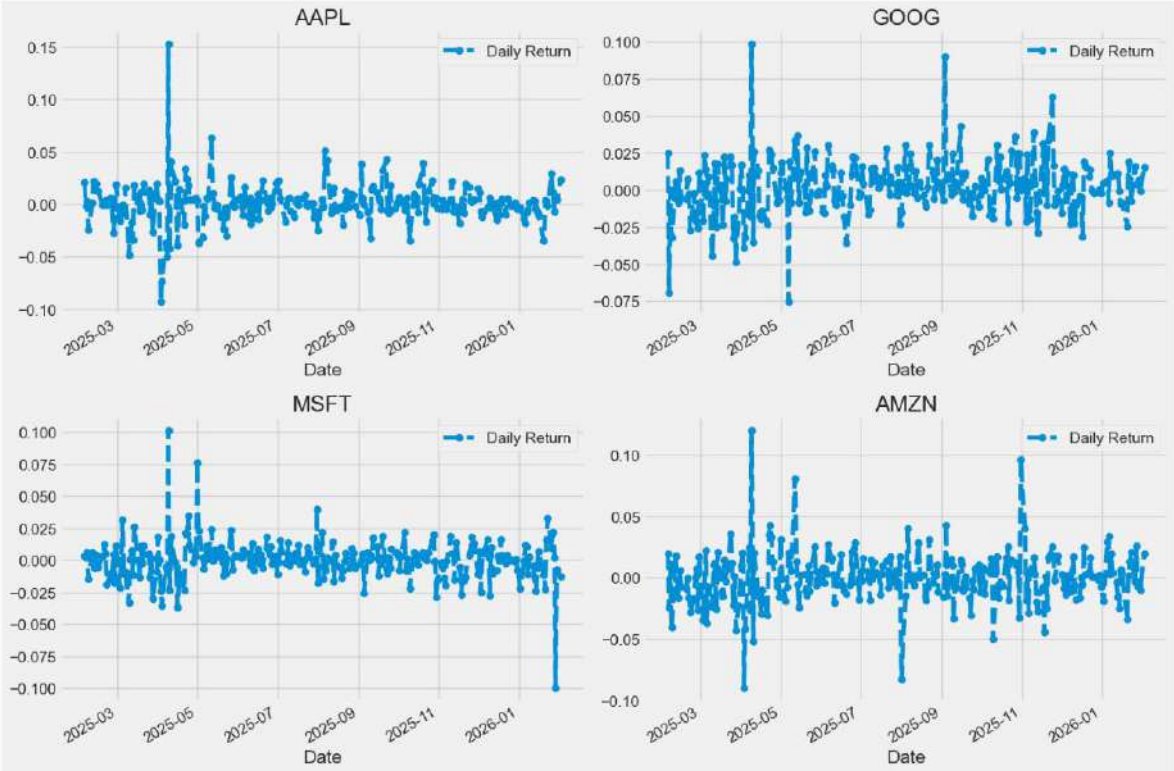
In [20]:

```
plt.figure(figsize=(12, 12))
plt.subplot(2, 2, 1)
sns.heatmap(tech_rets.corr(), annot=True, cmap='viridis')
plt.title('Correlation of stock return')
plt.subplot(2, 2, 2)
sns.heatmap(closing_df.corr(), annot=True, cmap='viridis')
plt.title('Correlation of stock closing price')
plt.show()
```




Risk vs Return of Tech Stocks
AMZN
AAPL
GOOG
MSFT
Risk (Standard Deviation)
0.022
0.021
0.020
0.019
0.018
0.017
0.016
0.00025
0.00050
0.00075
0.00100
0.00125
0.00150
0.00175
0.00200
0.00225
Expected Return


APPLE
GOOGLE
MICROSOFT
AMAZON
Counts
Daily Return



Model Building (LSTM)

Model Used:

Long Short-Term Memory (LSTM)

Why LSTM?

```
In [39]: from keras.models import Sequential
         from keras.layers import Dense, LSTM

         model = Sequential()

         # Smaller LSTM layers
         model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
         model.add(LSTM(25, return_sequences=False))

         model.add(Dense(10))
         model.add(Dense(1))

         model.compile(optimizer='adam', loss='mean_squared_error')

         model.fit(
             X_train,
             y_train,
             batch_size=32,
             epochs=5,
             verbose=1
         )

         C:\Users\range\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model
         instead.
           super().__init__(**kwargs)
         Epoch 1/5
         104/104 ━━━━━━━━━━━━━━━━━━━━ 7s 22ms/step - loss: 0.0030
         Epoch 2/5
         104/104 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - loss: 2.4041e-04
         Epoch 3/5
         104/104 ━━━━━━━━━━━━━━━━━━━━ 6s 30ms/step - loss: 2.3418e-04
         Epoch 4/5
         104/104 ━━━━━━━━━━━━━━━━━━━━ 5s 39ms/step - loss: 2.3764e-04
         Epoch 5/5
         104/104 ━━━━━━━━━━━━━━━━━━━━ 5s 24ms/step - loss: 2.2264e-04
Out[39]: <keras.src.callbacks.history.History at 0x2da476be800>
```

Handles time-series data effectively

Captures long-term dependencies

Performs better than traditional models for sequential data

LSTM is a type of recurrent neural network designed for sequence prediction tasks like stock forecasting

Model Training

Trained LSTM model on historical stock data

Used past time steps to predict future prices

Optimized using appropriate loss function (e.g., MSE)

Model Evaluation

Compared predicted values with actual stock prices

Out[48]:

Ticker	AAPL	Predictions
Date		
2025-05-20	206.425400	207.640472
2025-05-21	201.665421	208.030380
2025-05-22	200.936951	207.886887
2025-05-23	194.859741	207.414642
2025-05-27	199.789368	206.358109
...	...	...
2026-01-27	258.269989	251.951218
2026-01-28	256.440002	251.809402
2026-01-29	258.279999	251.887070
2026-01-30	259.480011	252.239243
2026-02-02	265.500000	252.800888

177 rows × 2 columns

Results:

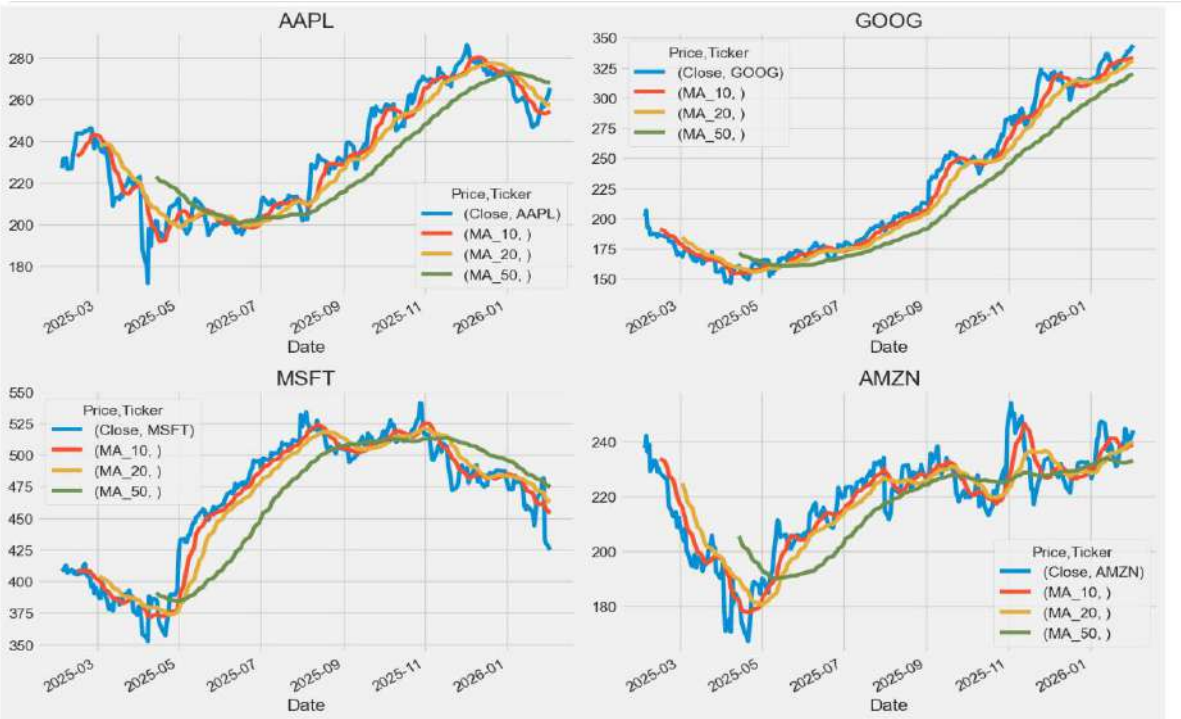
- Model successfully captured overall trends in stock prices
- Predictions closely followed actual values (trend-wise)

Visualization

Stock price trend graphs

Training vs prediction plots

Actual vs predicted price comparison



Conclusion

Successfully implemented a time-series forecasting model using LSTM

Data analysis helped understand stock behavior and trends

LSTM proved effective in capturing temporal dependencies in stock data

Future Scope

Use more advanced models (GRU, Transformer)

Include additional features (news sentiment, indicators)

Improve accuracy with hyperparameter tuning

Deploy as a real-time stock prediction app